

VLA 적용 가능성 간이 실험 로드맵

- 1 모델
- 2 실험
 - 2.1 환경 세팅
 - 2.2 데모 데이터 수집
 - 2.3 학습
 - 2.4 양자화 (Optimization)
- 3 평가 지표

모델

선택 모델: Octo, LeRobot (ACT, DiffusionPolicy)

선택 이유:

- 언어 지시 불필요, 시연 데이터만으로 학습
- 11GB VRAM에서 양자화 없이 바로 돌아감
- RealSense D435if 이미 있어서 파이프라인 연결 쉬움

실험

환경 세팅

```
1 # create venv
2 python3 -m venv ~/venv/robot
3 source ~/venv/robot/bin/activate
4
5 # PyTorch (CUDA 12.0)
6 pip install torch torchvision --index-url
7 https://download.pytorch.org/whl/cu121
8
9 # YOLO
10 pip install ultralytics
11
12 # LeRobot
13 pip install lerobot
14
15 # LeRobot 설치
16 pip install lerobot
17
18 # RealSense 연결 확인
19 pip install pyrealsense2
20
21 # ROS2 bridge (Robot Arm 제어용)
22 pip install lerobot[ros2]
```

- 확인할 것:
 - RealSense D435if → RGB + Depth 동시 스트림 테스트
 - Robot Arm ↔ ROS2 ↔ LeRobot 토픽 연결

데모 데이터 수집

목표: 50~100개 demonstration

항목	내용
방법	텔레오퍼레이션으로 직접 시연 녹화
입력	RGB-D 이미지 + 로봇 팔 joint 상태
출력	각 시점의 action (joint angle)
다양성	• 선반 높이 3~4 단계 • 좌우 위치 변화 포함

```
1 # LeRobot 데이터 수집 예시
2 from lerobot.common.datasets.lerobot_dataset import LeRobotDataset
3
4 dataset = LeRobotDataset.create(
5     repo_id="your_team/shelf_loading",
6     fps=30,
7     robot_type="your_arm"
8 )
```

학습

- RTX 2080 Ti 기준 약 2~4시간 소요

양자화 (Optimization)

ACT 자체는 ~80M params라 11GB에서 양자화 불필요하지만, 추론 속도 개선 목적으로는 적용 가능:

```
1 import torch
2 from torch.quantization import quantize_dynamic
3
4 # Dynamic Quantization (가장 간단)
5 model_quantized = quantize_dynamic(
6     model,
7     {torch.nn.Linear},
8     dtype=torch.qint8
9 )
10 # VRAM ~30% 절감, 속도 1.5~2x 향상
11
12 SmolVLA로 업그레이드 시엔 BitsAndBytes 사용:
13 from transformers import BitsAndBytesConfig
14
15 quantization_config = BitsAndBytesConfig(
16     load_in_8bit=True # 11GB → ~4GB로 절감
17 )
```

평가 지표

지 표	목 표
적 제 성 공 률	> 80%
위 치 오 차	< 10mm
추 론 시 간	< 100ms/frame